

CHAPITRE 1 : INTRODUCTION AUX SYSTÈMES EMBARQUÉS

PLAN DU CHAPITRE

Contenu :

- **Définitions fondamentales**
- **Applications des systèmes embarqués et IoT**
- **Tendances et marché**
- **Caractéristiques des systèmes embarqués**
- **Architecture des systèmes embarqués**
- **Contraintes et défis de conception**
- **Technologies de processeurs**
- **Circuits intégrés et cartes de développement**
- **Technologies de développement logiciel**
- **Multitâches et temps réel**
- **Outils de conception et déploiement**

CHAPITRE 1 : INTRODUCTION AUX SYSTÈMES EMBARQUÉS ET À L'INTERNET DES OBJETS

OBJECTIFS PÉDAGOGIQUES

Contenu :

À la fin de ce chapitre, vous serez capable de :

- ✓ Définir un système embarqué et l'IoT
- ✓ Identifier les domaines d'application
- ✓ Comprendre les tendances du marché
- ✓ Décrire l'architecture typique
- ✓ Expliquer les contraintes de conception
- ✓ Comparer les différentes technologies de processeurs
- ✓ Choisir une plateforme adaptée à un projet
- ✓ Connaître les outils de développement

1.1 DÉFINITIONS FONDAMENTALES

1.1.1 Qu'est-ce qu'un système embarqué ?

Un **système embarqué** (embedded system) est un système électronique et informatique autonome, spécialisé dans une tâche précise, et intégré physiquement dans un produit plus large. Contrairement à un ordinateur personnel polyvalent, un système embarqué est conçu pour exécuter une ou quelques fonctions dédiées, souvent avec des contraintes de ressources sévères.

Définition formelle : "Un système embarqué est un système microprogrammé intégré à un dispositif qu'il contrôle, avec lequel il interagit via des capteurs et actionneurs, et qui doit satisfaire des contraintes non-fonctionnelles strictes (temps réel, consommation, coût, fiabilité)."

1.1.2 Distinction fondamentale

Caractéristique	Système embarqué	Ordinateur personnel
Objectif	Dédié à une tâche spécifique	Polyvalent, multi-usages
Interface utilisateur	Minimale ou inexistante	Clavier, écran, souris
Système d'exploitation	Optionnel, temps réel possible	Généraliste (Windows, Linux, macOS)
Ressources	Limitées et optimisées	Abondantes et extensibles
Autonomie	Souvent critique	Secondaire
Fiabilité	Critique (parfois vitale)	Importante mais tolérante

1.1.3 Définition de l'Internet des Objets (IoT)

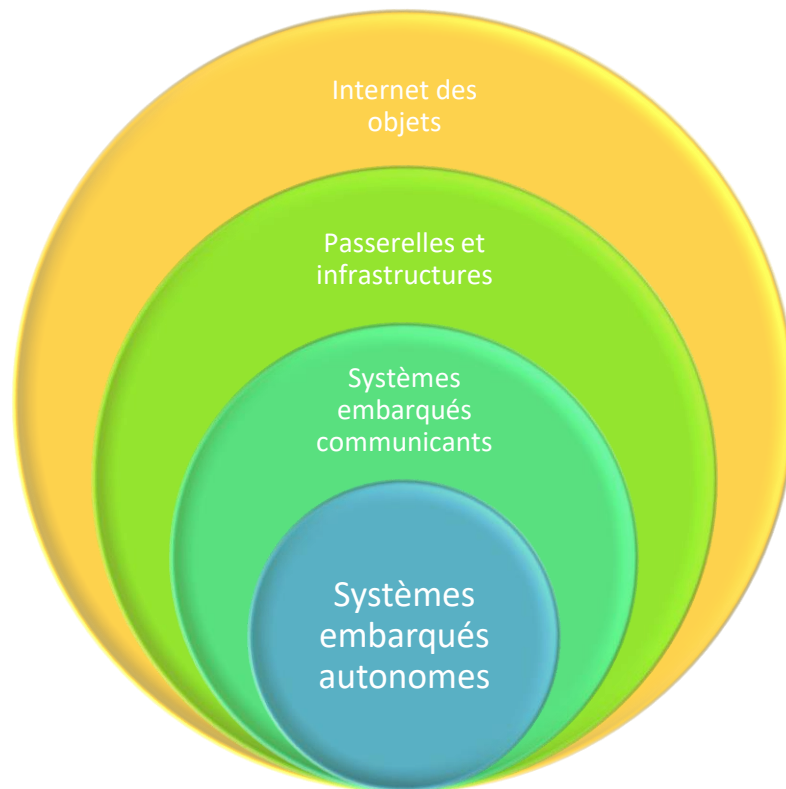
L'**Internet des Objets (IoT)** est l'extension du réseau Internet à des objets physiques ("choses") qui deviennent connectés et communicants. Ces objets sont équipés de capteurs, d'actionneurs et de capacités de traitement leur permettant de collecter, traiter et échanger des données avec d'autres objets ou avec des serveurs.

Formule conceptuelle :

IoT = Objets physiques + Capteurs/actionneurs + Connectivité + Traitement de données

1.1.4 Relation Systèmes Embarqués ↔ IoT

Les systèmes embarqués constituent le fondement technologique de l'IoT. Tout objet connecté est d'abord un système embarqué auquel on ajoute des capacités de communication réseau.



INTERNET DES OBJETS (Écosystème de données et services)
PASSERELLES ET INFRASTRUCTURE (Cloud, réseaux, plateformes)
SYSTÈMES EMBARQUÉS COMMUNICANTS (Objets connectés avec capteurs/actionneurs)
SYSTÈMES EMBARQUÉS AUTONOMES (Sans communication externe)

1.2 APPLICATIONS DES SYSTÈMES EMBARQUÉS ET DE L'IOT

1.2.1 Domaines d'application des systèmes embarqués

Transport et automobile

- **Systèmes critiques** : ABS (Anti-lock Braking System), ESP (Electronic Stability Program), airbags
- **Confort** : Climatisation automatique, systèmes d'infodivertissement
- **Mobilité moderne** : Véhicules électriques, systèmes d'assistance à la conduite (ADAS)

Exemple chiffré : Une voiture moderne contient entre 50 et 100 microcontrôleurs, avec plus de 100 millions de lignes de code.

Aéronautique et spatial

- **Avionique :** Pilote automatique, systèmes de navigation
- **Contrôle moteur :** FADEC (Full Authority Digital Engine Control)
- **Satellites :** Contrôle d'attitude, télémesures

Équipements médicaux

- **Dispositifs implantables :** Stimulateurs cardiaques, pompes à insuline
- **Équipements hospitaliers :** Scanners IRM, moniteurs de signes vitaux
- **Diagnostic portable :** Glucomètres, tensiomètres

Électronique grand public

- **Domotique :** Thermostats intelligents, éclairage connecté
- **Multimédia :** Téléviseurs, consoles de jeu, enceintes connectées
- **Wearables :** Montres connectées, bracelets d'activité

Automatisation industrielle

- **Automates programmables (PLC) :** Contrôle de chaînes de production
- **Robotique :** Bras manipulateurs, robots mobiles
- **Contrôle de procédés :** Régulation température/pression

1.2.2 Applications spécifiques de l'IoT

Smart Home (Maison intelligente)

- **Sécurité :** Caméras connectées, détecteurs d'ouverture, alarmes
- **Gestion énergétique :** Thermostats intelligents (Nest), éclairage adaptatif
- **Électroménager connecté :** Réfrigérateurs, machines à laver, aspirateurs robots

Smart City (Ville intelligente)

- **Gestion du trafic :** Feux adaptatifs, stationnement intelligent
- **Environnement :** Surveillance qualité de l'air, gestion des déchets
- **Infrastructures :** Éclairage public connecté, compteurs intelligents

Industrie 4.0 / IIoT (Industrial IoT)

- **Maintenance prédictive** : Surveillance vibratoire des machines
- **Traçabilité** : RFID pour suivi de production
- **Jumeaux numériques** : Simulation temps réel des processus

Agriculture intelligente (Smart Farming)

- **Précision** : Irrigation optimisée par capteurs d'humidité
- **Surveillance** : Drones pour analyse des cultures
- **Élevage connecté** : Suivi sanitaire des troupeaux

Santé connectée (e-Health)

- **Télémédecine** : Consultation à distance avec dispositifs médicaux
- **Suivi chronique** : Patients diabétiques ou cardiaques
- **Aide à la personne** : Détection de chutes pour personnes âgées

1.3 TENDANCES ET MARCHÉ DE L'EMBARQUÉ

1.3.1 Évolution historique

1970s : Premiers microcontrôleurs (Intel 8048)

1980s : Généralisation dans l'automobile et l'industrie

1990s : Explosion des produits électroniques grand public

2000s : Connectivité et premiers objets communicants

2010s : Révolution IoT, convergence avec le cloud

2020s : Edge computing, intelligence artificielle embarquée

1.3.2 Chiffres clés du marché (2024)

- **Marché mondial des systèmes embarqués** : ~250 milliards USD
- **Croissance annuelle** : 6-8% (CAGR)
- **Microcontrôleurs vendus annuellement** : >30 milliards d'unités
- **Parts par secteur** :
 - Automobile : 35%
 - Industrie : 25%

- Électronique grand public : 20%
- Télécommunications : 10%
- Médical : 5%
- Autres : 5%

1.3.3 Tendances technologiques majeures

1. Edge Computing

Déplacement du traitement du cloud vers les périphéries du réseau, réduisant la latence et la bande passante.

2. Intelligence Artificielle embarquée (TinyML)

Exécution de modèles d'apprentissage automatique sur microcontrôleurs à faible consommation.

3. Sécurité renforcée

Intégration de modules de sécurité matériels (TPM, Secure Enclave) face à la multiplication des cyberattaques.

4. Convergence des technologies

Fusion des mondes IT (Information Technology) et OT (Operational Technology).

1.4 TENDANCES ET MARCHÉ DE L'IOT

1.4.1 Évolution et adoption

Phase 1 (2000-2010) : Connectivité basique, RFID, M2M (Machine to Machine)

Phase 2 (2010-2020) : Explosion des plateformes IoT, cloud, Big Data

Phase 3 (2020-présent) : Maturité, standardisation, IA à la périphérie

1.4.2 Statistiques du marché IoT

- **Objets connectés dans le monde** : >15 milliards (2024)
- **Projection 2030** : >30 milliards d'objets
- **Marché mondial IoT** : ~1 200 milliards USD d'ici 2027
- **Données générées** : 80 zettaoctets/an d'ici 2025 (zetta = 10^{21})

1.4.3 Segmentation du marché IoT

Segment	Part de marché	Applications principales
Industrie (IIoT)	35%	Maintenance prédictive, automatisation
Smart City	20%	Éclairage, stationnement, déchets
Santé	15%	Télésurveillance, wearables
Transport/Logistique	15%	Flottes, suivi conteneurs
Smart Home	10%	Domotique, sécurité
Agriculture	5%	Irrigation, surveillance

1.4.4 Tendances IoT émergentes

- **5G pour l'IoT** : Ultra faible latence, connectivité massive
- **Digital Twins** : Répliques virtuelles d'objets physiques
- **Blockchain pour IoT** : Traçabilité et sécurité décentralisée
- **IoT durable** : Objets à faible empreinte carbone

1.5 CARACTÉRISTIQUES DES SYSTÈMES EMBARQUÉS

1.5.1 Caractéristiques fonctionnelles

Spécificité de la tâche

Un système embarqué exécute un ensemble limité de fonctions prédéfinies, contrairement à un système généraliste.

Interaction avec l'environnement physique

- **Capteurs** : Acquisition de données du monde réel
- **Actionneurs** : Influence sur l'environnement

Temps réel

Capacité à fournir une réponse dans des délais temporels garantis.

1.5.2 Caractéristiques non-fonctionnelles

Fiabilité

- Disponibilité continue (parfois 24h/24, 7j/7)
- Durée de vie pouvant atteindre 10-20 ans

- Comportement prévisible en toutes circonstances

Sécurité fonctionnelle (Safety)

Protection contre les dommages physiques aux personnes et à l'environnement.

Sûreté (Security)

Protection contre les intrusions et les accès non autorisés.

Consommation électrique

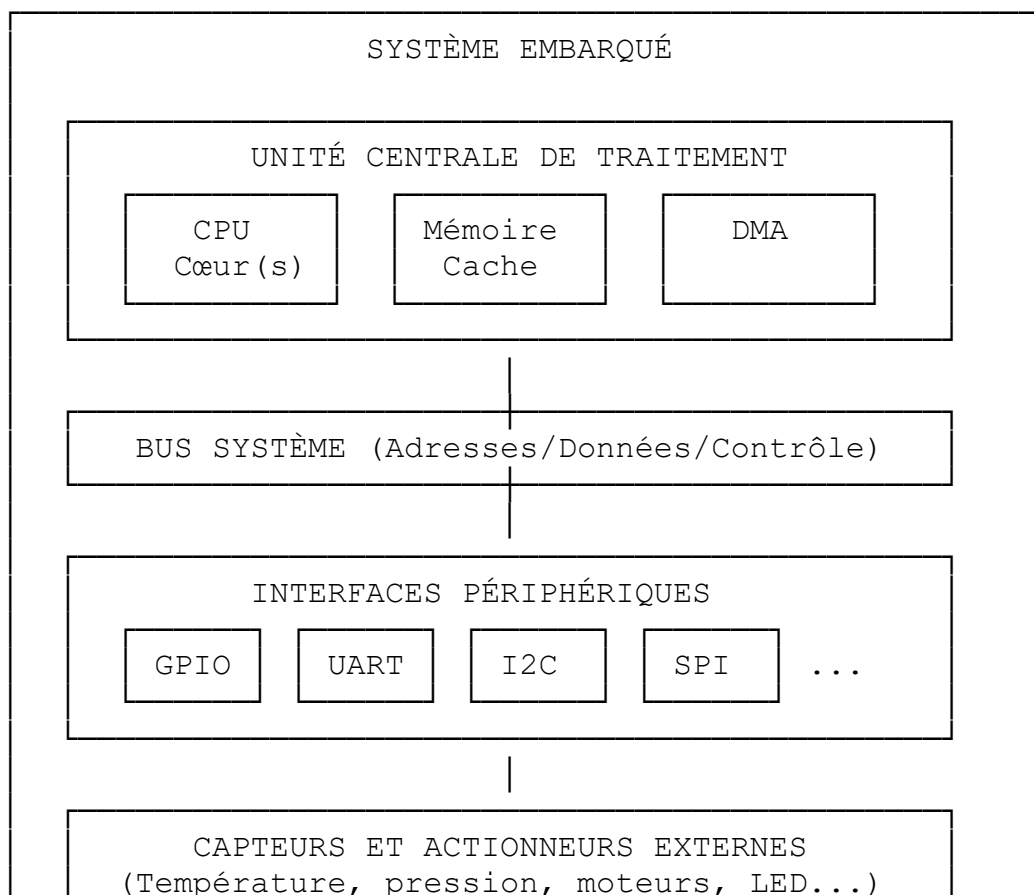
- Autonomie sur batteries (mois à années)
- Gestion d'énergie agressive

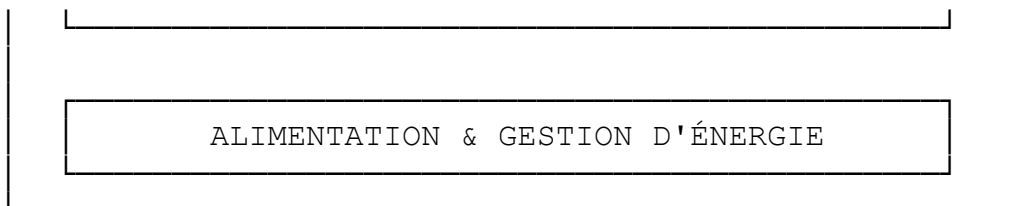
Coût

- Optimisation extrême pour les volumes
- Compromis entre performances et prix

1.6 ARCHITECTURE DES SYSTÈMES EMBARQUÉS

1.6.1 Architecture matérielle générale





1.6.2 Composants matériels clés

Processeur

- **Microcontrôleur (MCU)** : CPU, mémoire, périphériques intégrés
- **Microprocesseur (MPU)** : CPU seule, mémoire externe
- **DSP** : Traitement numérique du signal
- **FPGA** : Logique programmable

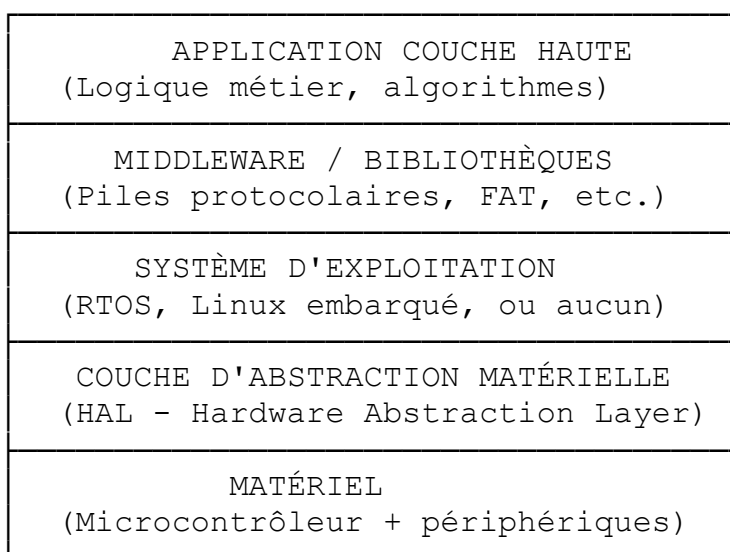
Mémoire

- **ROM/Flash** : Programme et données permanentes
- **RAM** : Données temporaires, pile, variables
- **EEPROM** : Stockage non-volatile de petite taille ; paramètres et préférences

Périphériques intégrés

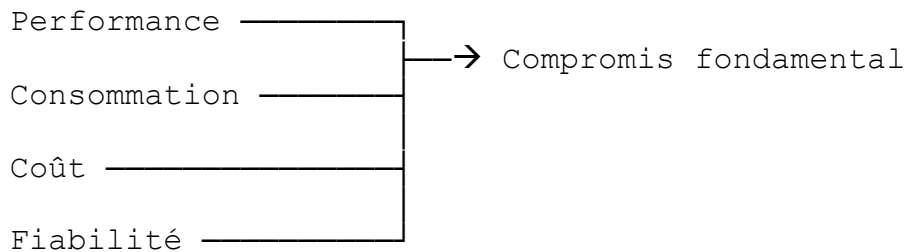
- **Timers/Compteurs** : Mesure du temps, PWM
- **Convertisseurs** : ADC (Analog to Digital), DAC (Digital to Analog)
- **Interfaces de communication** : UART, I2C, SPI, CAN, USB

1.6.3 Architecture logicielle



1.7 CONTRAINTES ET DÉFIS : OPTIMISATION DES MÉTRIQUES

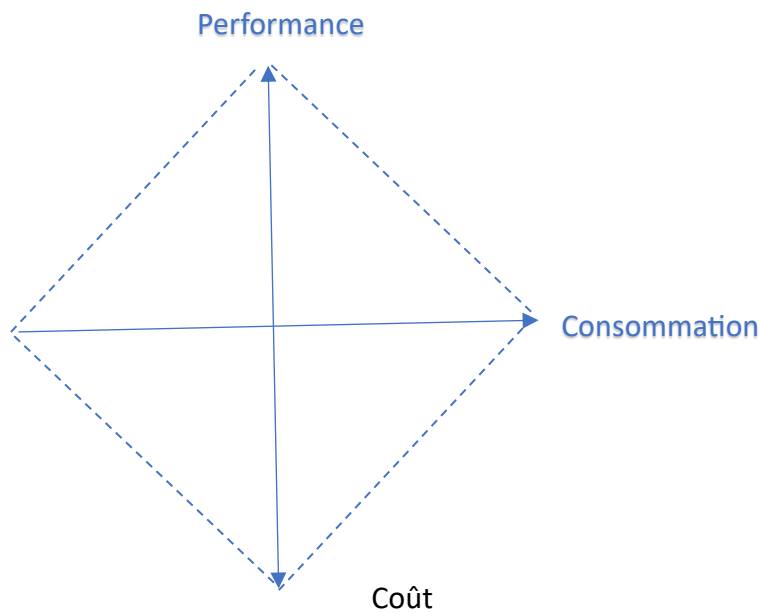
1.7.1 Les métriques fondamentales à optimiser



1.7.2 Matrice des contraintes

Contrainte	Description	Techniques d'optimisation
Performance	Vitesse d'exécution, temps de réponse	Algorithmes efficaces, matériel dédié, cache
Consommation	Puissance électrique, autonomie	Modes veille, horloge variable, basse tension
Coût	Prix unitaire, NRE (Non-Recurring Engineering)	Intégration, choix composants, volume
Taille/Encombrement	Empreinte physique	Intégration, SoC, PCB compact
Fiabilité	MTBF (Mean Time Between Failures), robustesse	Redondance, tests, marges de conception
Sécurité	Protection données, résistance attaques	Cryptographie, isolation matérielle
Temps de développement	Time-to-market	Réutilisation, outils, plateformes standards

1.7.3 Le cube des compromis (Design Cube)



1.7.4 Défis spécifiques à l'IoT

1. **Interopérabilité** : Communication entre objets hétérogènes
2. **Passage à l'échelle** : Gestion de millions d'objets
3. **Sécurité** : Vulnérabilités des objets connectés
4. **Confidentialité** : Protection des données personnelles
5. **Gestion du cycle de vie** : Mises à jour, obsolescence

1.8 TECHNOLOGIES DE PROCESSEURS POUR L'EMBARQUÉ ET L'IOT

1.8.1 Classification des processeurs embarqués

Par architecture

- **ARM** : Domination du marché mobile et IoT (Cortex-M, Cortex-A)
- **RISC-V** : Architecture ouverte émergente
- **x86** : Présent dans certains systèmes embarqués complexes
- **Xtensa** : ESP32, configurable
- **AVR** : Arduino, 8 bits
- **PIC** : Microchip, large gamme

Par puissance/capacité

Catégorie	Bits	Fréquence	RAM	Applications
Ultra low-power	8/16	<50 MHz	<64 KB	Capteurs, wearables
Milieu de gamme	32	50-400 MHz	64 KB - 1 MB	IoT, contrôle
Haute performance	32/64	>400 MHz	>1 MB	Multimédia, passerelles
MPU	32/64	>1 GHz	>256 MB	Linux embarqué

1.8.2 Principales familles pour l'IoT

Famille	Architecture	Avantages	Cas d'usage
ARM Cortex-M0/M3/M4	32-bit ARM	Écosystème riche, faible consommation	IoT grand public
ESP32	Xtensa LX6	WiFi/BT intégré, prix bas	Objets connectés
STM32	ARM Cortex-M	Large gamme, périphériques	Industrie, médical
Raspberry Pi (Broadcom)	ARM Cortex-A	Linux, puissance	Passerelles, IHM
Nordic nRF52	ARM Cortex-M4	BLE optimisé	Wearables, balises Bluetooth
SiFive (RISC-V)	RISC-V	Open source, personnalisable	Éducation, niche

1.9 TECHNOLOGIES DE CIRCUITS INTÉGRÉS ET CARTES DE DÉVELOPPEMENT

1.9.1 Niveaux d'intégration

ASIC	(Coût NRE élevé, volume)
SoC	
SiP (System in Package)	
Module (SOM)	
Carte de développement	(Flexibilité, prototypage)

1.9.2 Cartes de développement populaires

Cartes grand public

Carte	Processeur	Connectivité	Prix (FCFA)	Idéal pour
Arduino Uno	ATmega328P	Filaire	7 000	Débutants, éducation
ESP32 DevKit	ESP32	WiFi/BT	5 000	IoT, prototypage
Raspberry Pi 4	Cortex-A72	WiFi/Ethernet	70 000	Projets complexes
STM32 Nucleo	STM32	Optionnelle	10 000	Industrie
BBC micro:bit	nRF52	BLE	10 000	Éducation

Cartes professionnelles

- NXP [i.MX](#) Series : Applications multimédia

- **Texas Instruments Sitara** : Industrie
- **NVIDIA Jetson** : IA embarquée
- **Intel NUC** : PC embarqué

1.9.3 Tendances en intégration

- **SoC (System on Chip)** : Intégration toujours plus poussée
- **SiP (System in Package)** : Plusieurs puces dans un même boîtier
- **Hétérogénéité** : CPU + GPU + NPU (Neural Processing Unit) sur une même puce
- **3D IC** : Empilement de puces

1.10 TECHNOLOGIES DE DÉVELOPPEMENT LOGICIEL

1.10.1 Modèle en couches

APPLICATION	
MIDDLEWARE	Piles protocolaires (TCP/IP, MQTT) Systèmes de fichiers (FAT, LittleFS) Gestionnaires de périphériques
HAL	Abstraction du matériel
DRIVERS	Pilotes spécifiques

Avantages du modèle en couches :

- Portabilité
- Réutilisabilité
- Maintenabilité
- Testabilité

1.10.2 Modèle orienté objet pour l'embarqué

Bien que les contraintes mémoire limitent parfois l'usage intensif de l'OOP, des approches adaptées existent :

cpp

// Exemple C++ allégé pour embarqué

```
class Capteur {
```

```
protected:
```

```

    int pin;

    float valeur;

public:
    Capteur(int p) : pin(p) {}

    virtual void lire() = 0; // Polymorphisme

    float getValeur() { return valeur; }

};

```

```

class CapteurTemperature : public Capteur {
public:
    CapteurTemperature(int p) : Capteur(p) {}

    void lire() override {
        // Lecture spécifique température
    }

};

```

```

class CapteurHumidite : public Capteur {
public:
    CapteurHumidite(int p) : Capteur(p) {}

    void lire() override {
        // Lecture spécifique humidité
    }

};

```

1.10.3 Modèle par tâches

Organisation du logiciel comme un ensemble de tâches concurrentes :

```

// Pseudo-code d'architecture par tâches

TACHE tache_lecture_capteurs() {

```

```

while(1) {
    lire_temperature();
    lire_pression();
    attendre(100ms); // Périodique
}
}

```

```

TACHE tache_traitement_donnees() {
    while(1) {
        if (nouvelle_donnee_disponible()) {
            traiter_donnees();
            mettre_a_jour_affichage();
        }
        attendre_evenement();
    }
}

```

```

TACHE tache_communication() {
    while(1) {
        if (donnee_a_transmettre()) {
            envoyer_donnees();
        }
        attendre(1000ms); // Périodique
    }
}

```

1.11 MULTITÂCHES ET TEMPS RÉEL

1.11.1 Concepts fondamentaux

Temps réel

Capacité d'un système à garantir des délais de réponse déterminés.

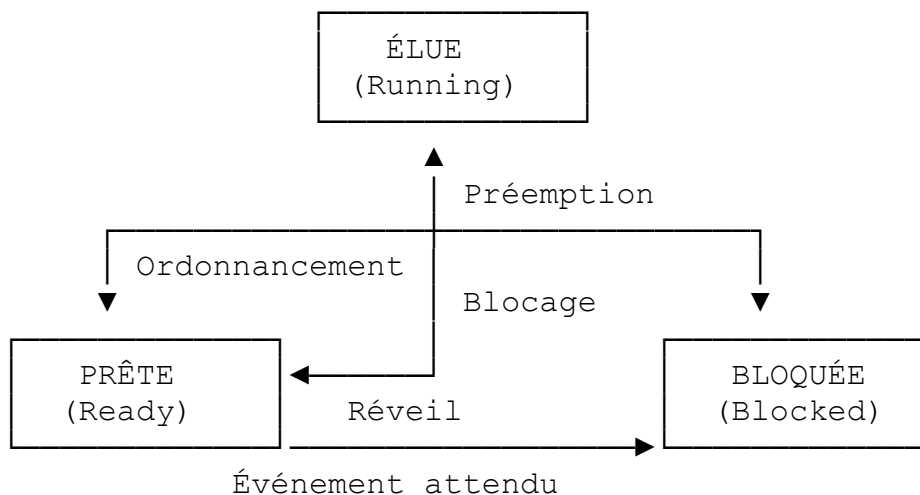
- **Temps réel dur** : Non-respect = défaillance catastrophique (airbag, freinage)
- **Temps réel ferme** : Tolérance limitée avec dégradation acceptable
- **Temps réel souple** : Performance moyenne suffisante

Ordonnancement

Mécanisme de répartition du temps CPU entre les tâches.

Algorithme	Principe	Utilisation
Rate Monotonic (RMS)	Priorité fixe = fréquence	Temps réel dur classique
Earliest Deadline First (EDF)	Priorité dynamique par échéance	Optimisation théorique
Round Robin	Tourniquet temporel	Systèmes non temps réel

1.11.2 États d'une tâche

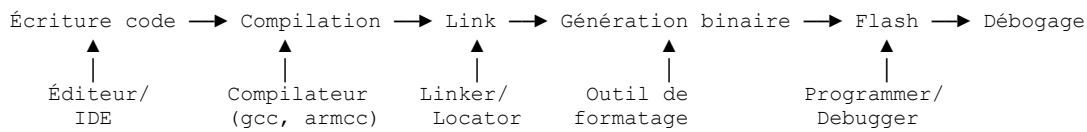


1.11.3 Mécanismes de synchronisation

- **Sémaphores** : Gestion de ressources partagées
- **Mutex** : Exclusion mutuelle avec prévention des inversions de priorité
- **Queues/Boîtes aux lettres** : Communication inter-tâches
- **Événements** : Signalisation entre tâches

1.12 OUTILS DE CONCEPTION ET DE DÉPLOIEMENT

1.12.1 Chaîne de développement typique



1.12.2 Outils par phase de développement

Modélisation et conception

- **UML** : Enterprise Architect, Visual Paradigm, Papyrus
- **Modélisation système** : MATLAB/Simulink, Scade
- **Architecture** : MagicDraw, Rational Rhapsody

Développement

- **IDE** : Eclipse, VS Code, Arduino IDE, PlatformIO, IAR, Keil
- **Compilateurs** : GCC (ARM, AVR), Clang, IAR, ARMCC
- **Gestion versions** : Git, SVN

Débogage et test

- **Débogueurs** : GDB, OpenOCD, Lauterbach
- **Analyseurs logiques** : Saleae, Logic
- **Oscilloscopes** : Tektronix, Keysight
- **Tests unitaires** : Unity, Ceedling, CMock

Déploiement et maintenance

- **Programmateurs** : ST-Link, J-Link, AVR ISP
- **Bootloaders** : Mise à jour à distance
- **OTA (Over-The-Air)** : ESP OTA, AWS IoT OTA
- **Monitoring** : Grafana, ThingsBoard

1.12.3 Environnements de développement intégrés

IDE	Plateformes supportées Particularités	
Arduino IDE	Arduino, ESP32, etc.	Simplicité, débutants
PlatformIO	Multi-plateformes	Professionnel, VS Code

IDE	Plateformes supportées	Particularités
STM32CubeIDE	STM32	Intégration complète ST
MPLAB X	Microchip PIC	Spécifique Microchip
IAR Embedded Workbench	Multi	Industriel, optimisation
Keil MDK	ARM	Référence ARM

1.13 SYNTHÈSE DU CHAPITRE

Points clés à retenir

1. **Définition** : Un système embarqué est un système informatique dédié intégré dans un produit.
2. **IoT** : Extension de l'Internet aux objets physiques via des systèmes embarqués communicants.
3. **Applications** : Présentes dans tous les secteurs (transport, santé, industrie, domotique).
4. **Caractéristiques** : Spécificité, interaction physique, contraintes multiples (temps réel, consommation, coût).
5. **Architecture** : Hiérarchie matérielle et logicielle avec couches d'abstraction.
6. **Compromis** : Optimisation entre performance, consommation, coût et fiabilité.
7. **Technologies** : Large gamme de processeurs (ARM dominant) et cartes de développement.
8. **Développement** : Modèles structurés (couches, tâches) et outils spécialisés.

Questions de réflexion

1. Quelles sont les différences fondamentales entre un système embarqué et un ordinateur personnel ?
2. Comment l'IoT transforme-t-il les industries traditionnelles ?
3. Pourquoi le compromis performance/consommation est-il critique dans les objets connectés ?
4. Quels défis supplémentaires apporte la connectivité dans un système embarqué ?
5. Comment choisir la bonne plateforme matérielle pour un projet donné ?

Exercice d'application : Identifiez 5 systèmes embarqués dans votre environnement quotidien. Pour chacun, décrivez :

- La fonction principale
 - Les capteurs/actionneurs probables
 - Les contraintes principales (consommation, temps réel, coût)
 - La nécessité (ou non) de connectivité
-

